

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Вычислительная математика»
Тема: Метод хорд

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Нахождение корня уравнения методом хорд с заданной точностью Eps, исследование зависимости числа итераций от точности Eps при заданном изменении Eps, исследование чувствительности к ошибкам в исходных данных.

Задание.

Используя программы - функции HORDA и Round из файла methods.cpp, найти корень уравнения с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Вариант 17: $f(x) = e^x - \arccos(\sqrt{x})$

Порядок выполнения:

1) Графически или аналитически отделить корень уравнения (т.е. найти отрезки [Left, Right], на которых функция удовлетворяет условиям применимости метода).

2) Составить подпрограмму - функцию вычисления функции, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Теоретические положения.

Пусть найден отрезок [a, b], на котором функция $f(x)$ меняет знак. Для определенности положим $f(a) > 0$, $f(b) < 0$. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения $f(x) = 0$

принимаются значения $c_0, c_1, \dots$ точек пересечения хорды с осью абсцисс, как это показано на рис.1.

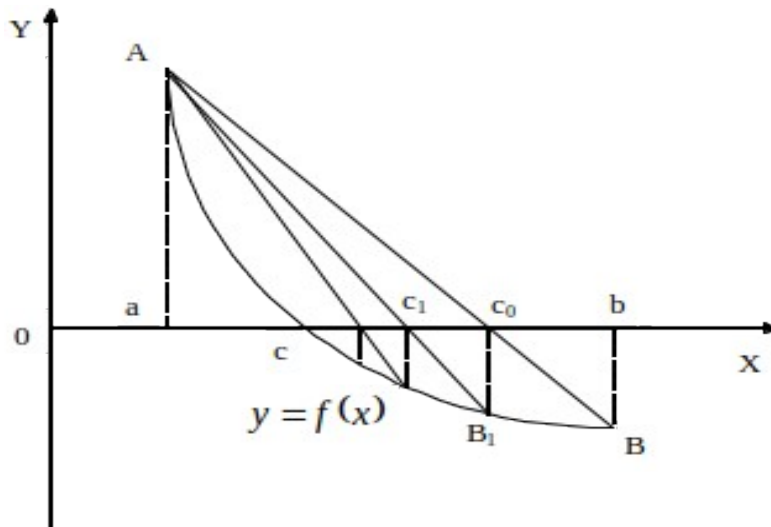


Рис. 1

Сначала находится уравнение хорды АВ:

$$\frac{y - f(a)}{f(b) - f(a)} = \frac{x - a}{b - a}.$$

Для точки пересечения ее с осью абсцисс ($x=c_0, y=0$) получается уравнение

$$c_0 = a - \frac{b - a}{f(b) - f(a)} f(a).$$

Далее сравниваются знаки величин $f(a)$ и $f(c_0)$ и для рассматриваемого случая оказывается, что корень находится в интервале (a, c_0) , так как $f(a)f(c_0) < 0$. Отрезок $[c_0, b]$ отбрасывается. Следующая итерации состоит в определении нового приближения c_1 как точки пересечения хорды AB_1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение $f(c_n)$ не станет по модулю меньше заданного числа ε .

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

Выполнение работы.

1) Корень уравнения $f(x) = 0$ для $f(x) = e^x - \arccos(\sqrt{x})$ - был найден графически при помощи графического калькулятора Desmos. Полученный график представлен на рисунке 1.

Рассмотрим отрезок $[0.154405, 0.154407]$. Функция монотонно убывающая. Из графика видно, что $f(0.154405) < 0$, $f(0.154407) > 0$. Следовательно на данном отрезке функция удовлетворяет условиям Коши.

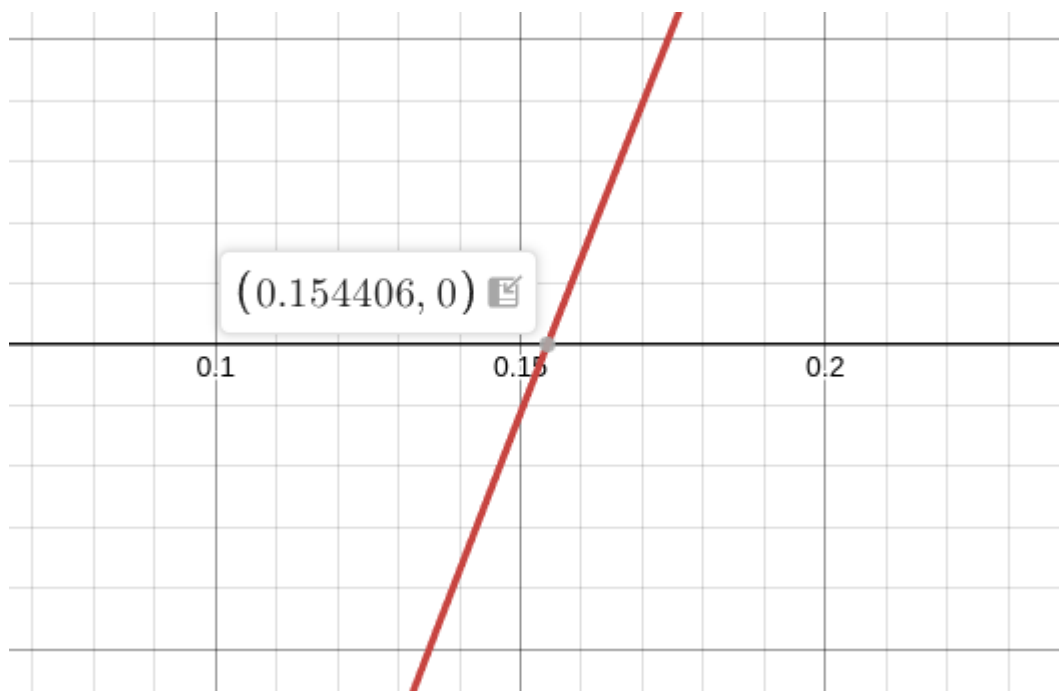


Рисунок 1 — График функции $f(x) = e^x - \arccos(\sqrt{x})$

2) `double F(double x)` — функция, принимающая значение x и возвращающая значение функции $f(x) = e^x - \arccos(\sqrt{x})$

Глобальная переменная `CurrentDelta` отвечает за значение погрешности. Если она больше нуля, то в функции `F` вычисленное значение функции $f(x) = e^x - \arccos(\sqrt{x})$ округляется при помощи функции `Round` с заданной точностью.

3) В функции `main` задаются начальные значения границ отрезка: $[0.1, 0.2]$. Данные значения удовлетворяют условиям Коши, а также получившийся подотрезок больше $2 * \text{Eps}$.

4) Затем в цикле выполняется функция `HORDA` для значений точности `Eps` от 0.1 до 0.000001. Полученные результаты выводятся. Данный вывод

получает на вход файл plots.py. В данном файле при помощи python-модуля matplotlib строится график зависимости числа итераций от точности Eps. Для наглядности шкала по оси абсцисс — логарифмическая. Из графика хорошо видно, что рост количества итераций при увеличении точности Eps — экспоненциальный.

Сами измерения представлены в таблице 1.

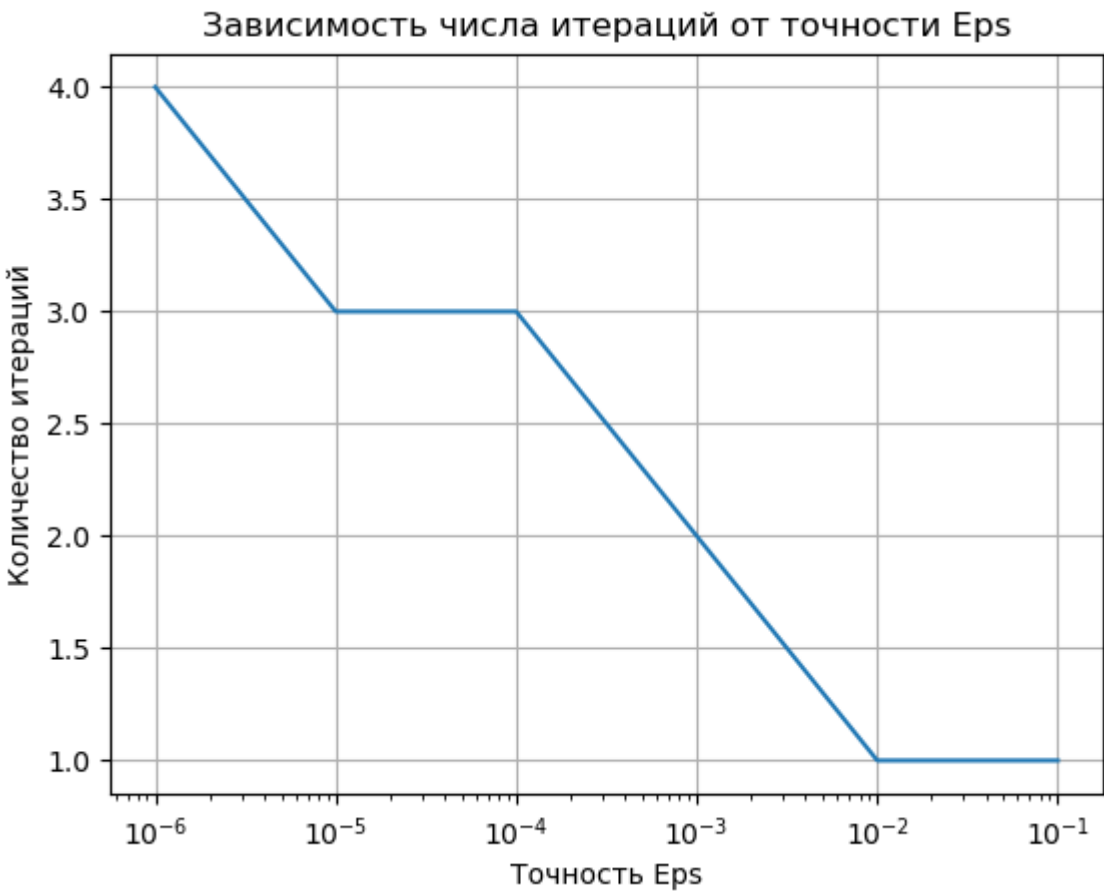


Рисунок 2 — График зависимости числа итераций от точности Eps

Таблица 1 — Исследование зависимости числа итераций от точности Eps

Точность Eps	Число итераций	Приблизженный корень
0.1	1	0.155737601362345
0.01	1	0.155737601362345
0.001	2	0.154452715601946
0.0001	3	0.154407434073878

0.00001	3	0.154407434073878
0.000001	4	0.154405828878976

5) Далее было произведено исследование чувствительности метода к ошибкам в исходных данных. Для этого была заведена глобальная переменная CurrentDelta, отвечающая за значение погрешности. Если она больше нуля, то в функции F вычисленное значение функции $f(x) = e^x - \arccos(\sqrt{x})$ округляется при помощи функции Round с заданной точностью.

В функции main() в цикле выполняется функция HORDA для погрешностей от 10^{-3} до 10^{-8} . Результаты измерений числа итераций и вычисленного корня подаются на вход файлу plots.py, в котором строится график зависимости вычисленного приближенного корня от погрешности Delta (см. рис. 3). Шкалы — линейные. Из графика видно, что рост точности вычисления корня зависит линейно от величины погрешности Delta и при минимальной погрешности совпадает с вычисленным ранее корнем. Полученные измерения представлены в таблице 2.

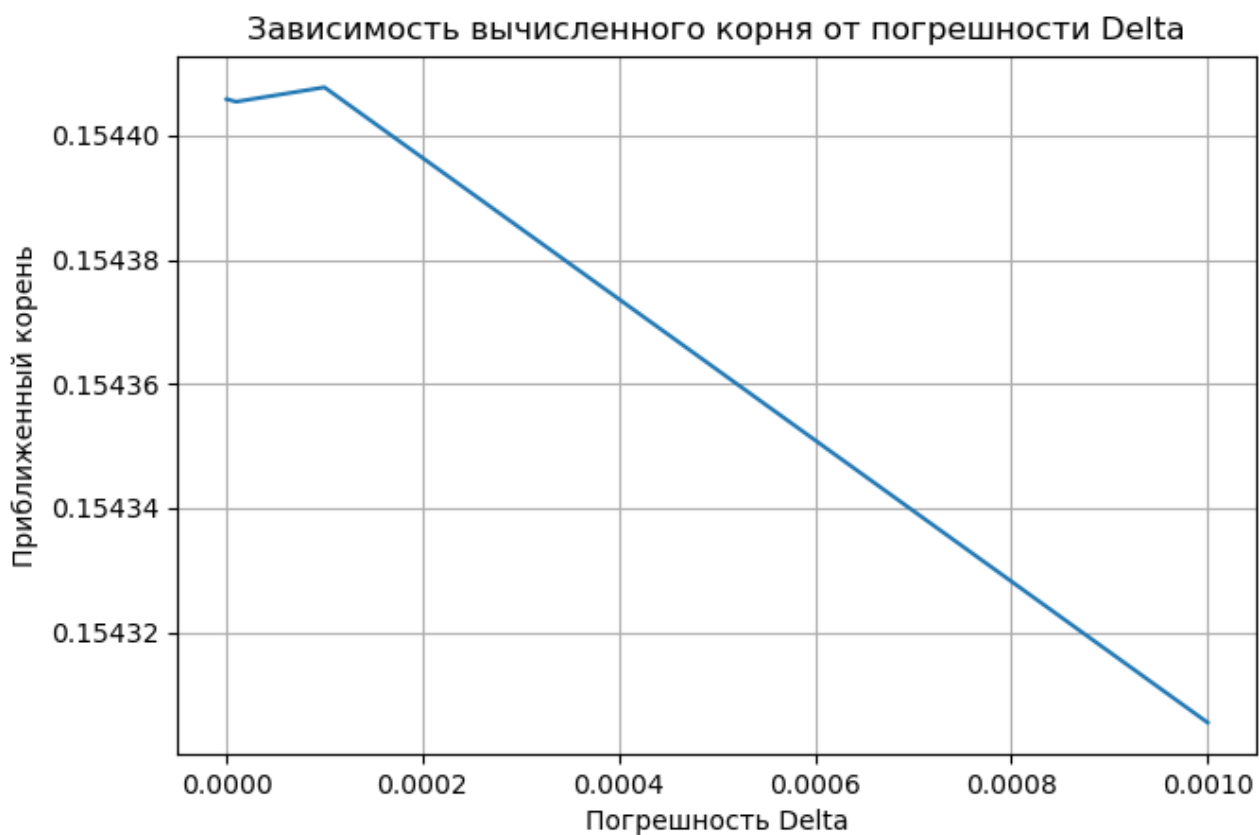


Рисунок 3 — Зависимость вычисленного корня от погрешности Delta

Таблица 2 — Исследование чувствительности к ошибкам исходных данных

Погрешность Delta	Число итераций	Приблизженный корень
0.001	1	0.154305468258957
0.0001	2	0.154407767370257
0.00001	3	0.154405453255742
0.000001	3	0.154405453255742
0.0000001	4	0.154405811989396
0.00000001	4	0.154405811989396

Разработанный программный код см. в приложении А.

Выводы.

В ходе лабораторной работы был найден корень уравнения методом хорд с заданной точностью ϵ_{ps} . Вычисленный результат совпал с результатом, вычисленным при помощи графического калькулятора. Было произведено исследование зависимости числа итераций от точности ϵ_{ps} при заданном изменении ϵ_{ps} , а также исследование чувствительности к ошибкам в исходных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb1.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

double CurrentDelta = 0.0;

double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return Delta * (long)((X / Delta) + 0.5);
    else
        return Delta * (long)((X / Delta) - 0.5);
}

double F(double x)
{
    double raw = exp(x) - acos(sqrt(x));
    if (CurrentDelta > 0.0)
        return Round(raw, CurrentDelta);
    else
        return raw;
}

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;

    if (FLeft * FRight > 0.0) {
        printf("Ошибка: знаки функции на концах [%lf, %lf]
одинаковы!\n", Left, Right);
        exit(1);
    }
    if (Eps <= 0.0) {
        puts("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    do {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);

        if (Y == 0.0) return X;
```

```

        if (Y * FLeft < 0.0) {
            Right = X;
            FRight = Y;
        }
        else {
            Left = X;
            FLeft = Y;
        }
        N++;
    } while (fabs(Y) >= Eps && N < 1000);

    return X;
}

int main()
{
    double Left = 0.1;
    double Right = 0.2;

    double EpsValues[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
    int numEps = sizeof(EpsValues) / sizeof(EpsValues[0]);

    printf("%d\n", numEps);
    CurrentDelta = 0.0;

    for (int i = 0; i < numEps; i++)
    {
        double eps = EpsValues[i];
        int iter;
        double root = HORDA(Left, Right, eps, iter);
        printf("%.15lf %d %.15lf\n", eps, iter, root);
    }

    double EpsFixed = 1e-6;
    double DeltaValues[] = {1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8};
    int numDelta = sizeof(DeltaValues) / sizeof(DeltaValues[0]);

    printf("%d\n", numDelta);

    for (int i = 0; i < numDelta; i++)
    {
        CurrentDelta = DeltaValues[i];
        int iter;
        double root = HORDA(Left, Right, EpsFixed, iter);
        printf("%.15lf %d %.15lf\n", CurrentDelta, iter, root);
    }

    return 0;
}

```

Название файла: plots.py

from matplotlib import pyplot as plt

```

def plot(x_data, y_data, x_label, y_label, name, file_name,
is_log_xscale = False):
    plt.plot(x_data, y_data)
    plt.grid()
    if is_log_xscale:
        plt.xscale('log')
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(name)
    plt.savefig(file_name)
    plt.show()

def main():
    eps_values = []
    n_values = []
    delta_values = []
    x_values = []

    print("Исследование зависимости числа операций от точности
eps");

    numEps = int(input())
    for _ in range(numEps):
        eps, n, x = [float(el) for el in input().split()]
        print(f"eps = {eps}, n = {n}, x = {x}")
        eps_values.append(eps)
        n_values.append(n)

        plot(eps_values, n_values, "Точность Eps", "Количество
итераций", "Зависимость числа итераций от точности Eps", "eps_n.png",
is_log_xscale=True)

    print("\nИсследование чувствительности к ошибкам исходных
данных")
    numDelta = int(input())
    for _ in range(numDelta):
        delta, n, x = [float(el) for el in input().split()]
        print(f"delta = {delta}, n = {n}, x = {x}")
        delta_values.append(delta)
        x_values.append(x)

        plot(delta_values, x_values, "Погрешность Delta", "Приближенный
корень", "Зависимость вычисленного корня от погрешности Delta",
"delta_x.png")

    if __name__ == "__main__":
        main()

```